

No **React Native**, “persistência” significa **salvar dados localmente no dispositivo**, para que eles continuem disponíveis mesmo após fechar o app.

Vou te mostrar **as formas mais usadas**, com **exemplos práticos**, começando pela mais comum em apps reais.

1 AsyncStorage (mais usado)

👉 Ideal para:

- Token de autenticação
- Preferências do usuário
- Dados simples (JSON)

📦 Instalação

```
npm install @react-native-async-storage/async-storage
```

📌 Salvando dados

```
import AsyncStorage from '@react-native-async-storage/async-storage';
```

```
async function salvarUsuario() {  
  const usuario = {  
    nome: 'João',  
    email: 'joao@email.com'  
  };  
  
  await AsyncStorage.setItem(  
    'usuario',  
    JSON.stringify(usuario)  
  );  
}
```

Lendo dados

```
async function buscarUsuario() {  
  const json = await AsyncStorage.getItem('usuario');  
  
  if (json !== null) {  
    const usuario = JSON.parse(json);  
    console.log(usuario.nome);  
  }  
}
```

Removendo dados

```
await AsyncStorage.removeItem('usuario');
```

Limpando tudo

```
await AsyncStorage.clear();
```

2 SecureStore (dados sensíveis)

 Ideal para:

- Senhas
- Tokens JWT

Expo

```
expo install expo-secure-store
```

Exemplo

```
import * as SecureStore from 'expo-secure-store';  
  
await SecureStore.setItemAsync('token', 'abc123');  
  
const token = await SecureStore.getItemAsync('token');
```

🔒 Mais seguro que AsyncStorage.

3 SQLite (dados estruturados)

👉 Ideal para:

- Apps offline
- Listas grandes
- Relacionamentos

📦 Expo:

```
expo install expo-sqlite
```

```
import * as SQLite from 'expo-sqlite';
```

```
let db;
```

```
async function getDB() {  
  if (!db) {  
    db = await SQLite.openDatabaseAsync('loja.db');  
  }  
  return db;  
}
```

```
export async function criarTabela() {  
  const db = await getDB();
```

```
  await db.execAsync(`  
    CREATE TABLE IF NOT EXISTS produtos (  
      id INTEGER PRIMARY KEY AUTOINCREMENT,  
      nome TEXT NOT NULL,  
      preco REAL NOT NULL  
    );  
  `);  
}
```

```

export async function listar() {
  const db = await getDB();
  return await db.getAllAsync('SELECT * FROM produtos;');
}

export async function inserir(nome, preco) {
  const db = await getDB();

  await db.runAsync(
    'INSERT INTO produtos (nome, preco) VALUES (?, ?);',
    [nome, preco]
  );
}

export async function atualizar(id, nome, preco) {
  const db = await getDB();

  await db.runAsync(
    'UPDATE produtos SET nome = ?, preco = ? WHERE id = ?;',
    [nome, preco, id]
  );
}

export async function excluir(id) {
  const db = await getDB();

  await db.runAsync(
    'DELETE FROM produtos WHERE id = ?;', [id] );
}

```

4 Qual usar?

Caso de uso	Melhor opção
Preferências	AsyncStorage
Token / senha	SecureStore

Dados offline	SQLite
Cache simples	AsyncStorage